

JoyWu_A02_CodingBasics.Rmd

Joy Wu

OVERVIEW

This exercise accompanies the lessons/labs in Environmental Data Analytics on coding basics.

Directions

1. Rename this file <FirstLast>_A02_CodingBasics.Rmd (replacing <FirstLast> with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. After Knitting, submit the completed exercise (PDF file) to Canvas.

Basics, Part 1

1. Generate a sequence of numbers from one to 55, increasing by fives. Assign this sequence a name.
2. Compute the mean and median of this sequence.
3. Ask R to determine whether the mean is greater than the median.
4. Insert comments in your code to describe what you are doing.

```
#1.Create a sequence of numbers from 1 to 55, increasing by 5. Assign this sequence a name.  
sequence <- seq(1,55,5)
```

```
#2.Compute the mean and median of this sequence.
```

```
sequence_mean <- mean(sequence)  
sequence_median <- median(sequence)
```

```
#3. Ask R to determine whether the mean is greater than the median.
```

```
if (sequence_mean > sequence_median){  
  print("The mean is greater than the median.")  
} else if (sequence_mean < sequence_median){print("The median is greater than the mean")  
} else {print("The mean is equal to the median.")}
```

```
## [1] "The mean is equal to the median."
```

Basics, Part 2

5. Create three vectors, each with four components, consisting of (a) student names, (b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).
6. Label each vector with a comment on what type of vector it is.
7. Combine each of the vectors into a data frame. Assign the data frame an informative name.
8. Label the columns of your data frame with informative titles.

```
#5&6. Create three vectors, each with four components, consisting of (a) student names,  
#(b) test scores, and (c) whether they are on scholarship or not (TRUE or FALSE).  
#Label each vector with a comment on what type of vector it is.  
student_names <- c("Amy","Ben","Cathy","Dylan") #character vector  
test_scores <- c(100,76,92,84) #numeric vector  
scholarship_status <- c(TRUE,FALSE,TRUE,TRUE) #logical vector  
  
#7&8. Combine each of the vectors into a data frame. Assign the data frame  
#an informative name. Label the columns of your data frame with informative titles.  
student_data <- data.frame(  
  Name = student_names,  
  Score = test_scores,  
  Scholarship = scholarship_status  
)
```

9. QUESTION: How is this data frame different from a matrix?

Answer: This data frame is different from a matrix in the sense that it can store different types of data. Like here, it contains numbers and characters. A matrix would only be able to store one type of data.

10. Create a function with one input. In this function, use `if...else` to evaluate the value of the input: if it is greater than 50, print the word “Pass”; otherwise print the word “Fail”.
11. Create a second function that does the exact same thing as the previous one but uses `ifelse()` instead of `if...else`.
12. Run both functions using the value 52.5 as the input
13. Run both functions using the **vector** of student test scores you created as the input. (Only one will work properly...)

```
#10. Create a function using if...else  
pass_fail_if <- function(x){  
  if (x>50){print("Pass")}  
  else{print("Fail")}  
}  
  
#11. Create a function using ifelse()  
pass_fail_ifelse <-function(x){  
  ifelse(x>50,"Pass","Fail")  
}  
  
#12a. Run the first function with the value 52.5  
pass_fail_if(52)
```

```
## [1] "Pass"
```

```
#12b. Run the second function with the value 52.5  
pass_fail_ifelse(52)
```

```
## [1] "Pass"
```

```
#13a. Run the first function with the vector of test scores (doesn't work)  
#pass_fail_if(test_scores)  
  
#13b. Run the second function with the vector of test scores  
pass_fail_ifelse(test_scores)
```

```
## [1] "Pass" "Pass" "Pass" "Pass"
```

14. QUESTION: Which option of `if...else` vs. `ifelse` worked? Why? (Hint: search the web for “R vectorization”)

Answer: The option “ifelse” worked. This is because `if...else` expects a single TRUE/FALSE condition. That is why I got an error saying “the condition has length >1”. However, `ifelse` is vectorized, meaning that it can evaluate the logical test for each element and returns a vector of the same length with results.

NOTE Before knitting, you’ll need to comment out the call to the function in Q13 that does not work. (A document can’t knit if the code it contains causes an error!)